

Basic Slicing and Advanced Indexing in NumPy Python

NumPy or Numeric Python is a package for computation on homogeneous n-dimensional arrays. In numpy dimensions are called as axes.

Why do we need NumPy ?

A question arises that why do we need NumPy when python lists are already there. The answer to it is we cannot perform operations on all the elements of two list directly. For example we cannot multiply two lists directly we will have to do it element wise. This is where the role of NumPy comes into play.

```
# Python program to demonstrate a need of NumPy

list1 = [1, 2, 3, 4 ,5, 6]
list2 = [10, 9, 8, 7, 6, 5]

# Multiplying both lists directly would give an error.
print(list1*list2)
Output :
```

TypeError: can't multiply sequence by non-int of type 'list'

Where as this can easily be done with NumPy arrays.

Another example,

```
# Python program to demonstrate the use of NumPy arrays
import numpy as np

list1 = [1, 2, 3, 4, 5, 6]
list2 = [10, 9, 8, 7, 6, 5]

# Convert list1 into a NumPy array
a1 = np.array(list1)

# Convert list2 into a NumPy array
a2 = np.array(list2)

print(a1*a2)
```

Output :

```
array([10, 18, 24, 28, 30, 30])
```

This article will help you get acquainted with indexing in NumPy in detail. Numpy package of python has a great power of indexing in different ways.

Indexing using index arrays

Indexing can be done in numpy by using an array as an index. In case of slice, a view or shallow copy of the array is returned but in index array a copy of the original array is returned. Numpy arrays can be indexed with other arrays or any other sequence with the exception of tuples. The last element is indexed by -1 second last by -2 and so on.

```
# Python program to demonstrate
# the use of index arrays.
import numpy as np

# Create a sequence of integers from 10 to 1 with a step of -2
a = np.arange(10, 1, -2)
print("\n A sequential array with a negative step: \n",a)

# Indexes are specified inside the np.array method.
newarr = a[np.array([3, 1, 2 ])]
print("\n Elements at these indices are:\n",newarr)
```

Output :

A sequential array with a negative step:

```
[10  8  6  4  2]
```

Elements at these indices are:

```
[4 8 6]
```

Another example,

```
import
numpy as np

# NumPy array with elements from 1 to 9
x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])

# Index values can be negative.
arr = x[np.array([1, 3, -3])]
print("\n Elements are : \n",arr)
Output :
```

Elements are:

```
[2 4 7]
```

Types of Indexing

There are two types of indexing :

1. Basic Slicing and indexing : Consider the syntax `x[obj]` where `x` is the array and `obj` is the index. Slice object is the index in case of basic slicing.

Basic slicing occurs when `obj` is :

1. a slice object that is of the form `start : stop : step`
2. an integer
3. or a tuple of slice objects and integers

All arrays generated by basic slicing are always view of the original array.

```
# Python program for basic slicing.
import numpy as np

# Arrange elements from 0 to 19
a = np.arange(20)
print("\n Array is:\n ",a)

# a[start:stop:step]
print("\n a[-8:17:1] = ",a[-8:17:1])

# The : operator means all elements till the end.
print("\n a[10:] = ",a[10:])
Output :
```

Array is:

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19]
```

```
a[-8:17:1] = [12 13 14 15 16]
```

```
a[10:] = [10 11 12 13 14 15 16 17 18 19]
```

Ellipsis can also be used along with basic slicing. Ellipsis (...) is the number of : objects needed to make a selection tuple of the same length as the dimensions of the array.

```
# Python program for indexing using basic slicing with ellipsis
import numpy as np
```

```
# A 3 dimensional array.
```

```
b = np.array([[[1, 2, 3],[4, 5, 6]],
              [[7, 8, 9],[10, 11, 12]]])
```

```
print(b[...,1]) #Equivalent to b[:, :, 1 ]
```

Output :

```
[[ 2  5]
```

```
 [ 8 11]]
```

2. Advanced indexing : Advanced indexing is triggered when obj is :

- an ndarray of type integer or Boolean
- or a tuple with at least one sequence object
- is a non tuple sequence object

Advanced indexing returns a copy of data rather than a view of it.

Advanced indexing is of two types integer and Boolean.

Purely integer indexing : When integers are used for indexing. Each element of first dimension is paired with the element of the second dimension. So the index of the elements in this case are (0,0),(1,0),(2,1) and the corresponding elements are selected.

- Python

```
# Python program showing advanced indexing
```

```
import numpy as np
```

```
a = np.array([[1 ,2 ],[3 ,4 ],[5 ,6 ]])
```

```
print(a[[0 ,1 ,2 ],[0 ,0 ,1]])
```

Output :

```
[1 3 6]
```

Boolean Indexing

This indexing has some boolean expression as the index. Those elements are returned which satisfy that Boolean expression. It is used for filtering the desired element values.

- Python

```
# You may wish to select numbers greater than 50
```

```
import numpy as np
```

```
a = np.array([10, 40, 80, 50, 100])
```

```
print(a[a>50])
```

Output :

```
[80 100]
```

```
# You may wish to square the multiples of 40
```

```
import numpy as np
```

```
a = np.array([10, 40, 80, 50, 100])
```

```
print(a[a%40==0]**2)
```

Output :

```
[1600 6400])
```

```
# You may wish to select those elements whose  
# sum of row is a multiple of 10.
```

```
import numpy as np
```

```
b = np.array([[5, 5],[4, 5],[16, 4]])
```

```
sumrow = b.sum(-1)
```

```
print(b[sumrow%10==0])
```

Output :

```
array([[ 5,  5], [16,  4]])
```

